

# Abstrakte Klasse `Client`, Vererbung und Beziehungen in Java

Benjamin Malicke ©

1. März 2026

## Inhaltsverzeichnis

|                                                                                                                 |           |
|-----------------------------------------------------------------------------------------------------------------|-----------|
| <b>1 Ziel dieses Beispiels</b>                                                                                  | <b>1</b>  |
| <b>2 Abstrakte Klasse <code>Client</code></b>                                                                   | <b>2</b>  |
| 2.1 Idee . . . . .                                                                                              | 2         |
| 2.2 Implementierung von <code>Client</code> (mit <code>protected</code> , ohne Umlaute im Code) . . . . .       | 2         |
| <b>3 Klasse <code>Wartungsvertrag</code> mit monatlichen Kosten</b>                                             | <b>3</b>  |
| 3.1 „hat eine“ statt Vererbung . . . . .                                                                        | 3         |
| 3.2 Implementierung von <code>Wartungsvertrag</code> (mit Kosten) . . . . .                                     | 3         |
| <b>4 Konkrete Unterklassen von <code>Client</code></b>                                                          | <b>4</b>  |
| 4.1 <code>Server</code> . . . . .                                                                               | 4         |
| 4.2 <code>Router</code> . . . . .                                                                               | 5         |
| 4.3 <code>Workstation</code> . . . . .                                                                          | 6         |
| 4.4 <code>VirtuellerDesktop</code> . . . . .                                                                    | 6         |
| <b>5 Nutzung in <code>main</code>: Array von <code>Client</code> und teuerster <code>Wartungsvertrag</code></b> | <b>7</b>  |
| <b>6 Erweiterungsaufgabe: <code>ThinClient</code></b>                                                           | <b>9</b>  |
| <b>7 Zusammenfassung der Konzepte (Profi-Version)</b>                                                           | <b>11</b> |
| <b>8 Prüfungsfragen</b>                                                                                         | <b>11</b> |
| <b>9 Musterlösungen</b>                                                                                         | <b>13</b> |
| <b>10 UML-Klassendiagramm (ASCII)</b>                                                                           | <b>16</b> |

## 1 Ziel dieses Beispiels

Wir modellieren ein Szenario aus der IT-Welt mit professionellem Anspruch:

- **Abstrakte Klasse `Client`** mit:
  - `leistung` (z.B. in Watt)
  - `ipAdresse`
  - `wartungsvertrag`
  - abstrakter Methode `berechneLeistungsaufnahme()`
- Konkrete Unterklassen:

- Server
  - Router
  - Workstation
  - VirtuellerDesktop
- „ist eine“-Beziehung (Vererbung): z.B. Server ist ein Client
  - „hat eine“-Beziehung (Komposition): z.B. Client hat einen Wartungsvertrag
  - In main: ein Array Client[] mit verschiedenen konkreten Objekten.
  - Algorithmus: teuersten aktiven Wartungsvertrag pro Monat finden.
  - Erweiterungsaufgabe: Implementieren eines ThinClient.

## 2 Abstrakte Klasse Client

### 2.1 Idee

Client beschreibt allgemein einen Netzwerkteilnehmer mit Leistungsaufnahme, IP-Adresse und optionalem Wartungsvertrag. Von Client sollen **keine direkten Objekte** erzeugt werden, nur spezialisierte Unterklassen.

Für Unterklassen verwenden wir hier protected-Attribute, um beispielhaft den direkten Zugriff auf gemeinsame Basisdaten zu zeigen.

### 2.2 Implementierung von Client (mit protected, ohne Umlaute im Code)

```

1 public abstract class Client {
2
3     // Basiseigenschaften fuer alle Clients
4     protected double leistung; // in Watt
5     protected String ipAdresse;
6     protected Wartungsvertrag wartungsvertrag;
7
8     public Client(double leistung, String ipAdresse,
9         Wartungsvertrag wartungsvertrag) {
10
11         // einfache Eingabepruefungen (Argument-Validation)
12         if (leistung <= 0) {
13             throw new IllegalArgumentException(
14                 "Leistung muss groesser als 0 sein");
15         }
16         if (ipAdresse == null || ipAdresse.isBlank()) {
17             throw new IllegalArgumentException(
18                 "ipAdresse darf nicht leer sein");
19         }
20
21         this.leistung = leistung;
22         this.ipAdresse = ipAdresse;
23         this.wartungsvertrag = wartungsvertrag;
24     }
25
26     public double getLeistung() {
27         return leistung;
28     }
29

```

```

30 public String getIpAdresse() {
31     return ipAdresse;
32 }
33
34 public Wartungsvertrag getWartungsvertrag() {
35     return wartungsvertrag;
36 }
37
38 // Beispiel fuer gemeinsame Funktionalitaet
39 public void einschalten() {
40     System.out.println(
41         "Client mit IP " + ipAdresse + " wird eingeschaltet...");
42 }
43
44 public void pruefeWartungsvertrag() {
45     if (wartungsvertrag != null && wartungsvertrag.istAktiv()) {
46         System.out.println(
47             "Wartungsvertrag " + wartungsvertrag.getVertragsNummer()
48             + " ist aktiv fuer IP " + ipAdresse);
49     } else {
50         System.out.println(
51             "Kein aktiver Wartungsvertrag fuer IP " + ipAdresse);
52     }
53 }
54
55 // Abstrakte Methode: jede Unterklasse muss definieren,
56 // wie die Leistungsaufnahme berechnet wird.
57 public abstract double berechneLeistungsaufnahme(
58     int betriebsstundenProTag);
59 }

```

### 3 Klasse Wartungsvertrag mit monatlichen Kosten

#### 3.1 „hat eine“statt Vererbung

Ein Client ist kein Wartungsvertrag, sondern **hat einen** Wartungsvertrag. Das ist eine klassische Komposition: Der Wartungsvertrag gehoert logisch zu einem Client.

#### 3.2 Implementierung von Wartungsvertrag (mit Kosten)

```

1 import java.time.LocalDate;
2
3 public class Wartungsvertrag {
4
5     private String vertragsNummer;
6     private LocalDate startDatum;
7     private LocalDate endDatum;
8     private double monatlicheKosten; // in Euro
9
10    public Wartungsvertrag(String vertragsNummer,
11        LocalDate startDatum,
12        LocalDate endDatum,
13        double monatlicheKosten) {
14
15        if (vertragsNummer == null || vertragsNummer.isBlank()) {
16            throw new IllegalArgumentException(

```

```

17  "vertragsNummer darf nicht leer sein");
18  }
19  if (startDatum == null || endDatum == null) {
20  throw new IllegalArgumentException(
21  "startDatum und endDatum duerfen nicht null sein");
22  }
23  if (endDatum.isBefore(startDatum)) {
24  throw new IllegalArgumentException(
25  "endDatum darf nicht vor startDatum liegen");
26  }
27  if (monatlicheKosten < 0) {
28  throw new IllegalArgumentException(
29  "monatlicheKosten duerfen nicht negativ sein");
30  }
31
32  this.vertragsNummer = vertragsNummer;
33  this.startDatum = startDatum;
34  this.endDatum = endDatum;
35  this.monatlicheKosten = monatlicheKosten;
36  }
37
38  public String getVertragsNummer() {
39  return vertragsNummer;
40  }
41
42  public double getMonatlicheKosten() {
43  return monatlicheKosten;
44  }
45
46  public boolean istAktiv() {
47  LocalDate heute = LocalDate.now();
48  return (heute.isEqual(startDatum) || heute.isAfter(startDatum))
49  && (heute.isBefore(endDatum) || heute.isEqual(endDatum));
50  }
51
52  // Erweiterungspunkt: hier koennte man spaeter weitere
53  // Kostenberechnungen einfuehren (z.B. jaehrliche Kosten).
54  }

```

## 4 Konkrete Unterklassen von Client

Alle folgenden Klassen **erben** von `Client`. Das zeigt die „ist eine“-Beziehung:

- `Server` ist ein `Client`
- `Router` ist ein `Client`
- `Workstation` ist ein `Client`
- `VirtuellerDesktop` ist ein `Client`

Jede Klasse implementiert die Methode `berechneLeistungsaufnahme` anders. So laesst sich Polymorphie anschaulich demonstrieren.

### 4.1 Server

```

1 public class Server extends Client {
2
3     private int anzahlCpus;
4
5     public Server(double leistung,
6         String ipAdresse,
7         Wartungsvertrag wartungsvertrag,
8         int anzahlCpus) {
9
10        super(leistung, ipAdresse, wartungsvertrag);
11
12        if (anzahlCpus <= 0) {
13            throw new IllegalArgumentException(
14                "anzahlCpus muss groesser als 0 sein");
15        }
16        this.anzahlCpus = anzahlCpus;
17    }
18
19    public int getAnzahlCpus() {
20        return anzahlCpus;
21    }
22
23    @Override
24    public double berechneLeistungsaufnahme(int betriebsstundenProTag) {
25        if (betriebsstundenProTag < 0 || betriebsstundenProTag > 24) {
26            throw new IllegalArgumentException(
27                "betriebsstundenProTag muss zwischen 0 und 24 liegen");
28        }
29        // vereinfachtes Beispiel:
30        // Server laeuft fast immer unter Last -> ca. 90% Nennleistung
31        double faktor = 0.9;
32        return leistung * faktor * betriebsstundenProTag;
33    }
34 }

```

## 4.2 Router

```

1 public class Router extends Client {
2
3     private int anzahlPorts;
4
5     public Router(double leistung,
6         String ipAdresse,
7         Wartungsvertrag wartungsvertrag,
8         int anzahlPorts) {
9
10        super(leistung, ipAdresse, wartungsvertrag);
11
12        if (anzahlPorts <= 0) {
13            throw new IllegalArgumentException(
14                "anzahlPorts muss groesser als 0 sein");
15        }
16        this.anzahlPorts = anzahlPorts;
17    }
18
19    public int getAnzahlPorts() {

```

```

20 | return anzahlPorts;
21 | }
22 |
23 | @Override
24 | public double berechneLeistungsaufnahme(int betriebsstundenProTag) {
25 | if (betriebsstundenProTag < 0 || betriebsstundenProTag > 24) {
26 | throw new IllegalArgumentException(
27 | "betriebsstundenProTag muss zwischen 0 und 24 liegen");
28 | }
29 | // Router haben meist einen relativ konstanten Verbrauch
30 | double faktor = 0.6;
31 | return leistung * faktor * betriebsstundenProTag;
32 | }
33 | }

```

### 4.3 Workstation

```

1 | public class Workstation extends Client {
2 |
3 |     private boolean hatGrafikkarte;
4 |
5 |     public Workstation(double leistung,
6 | String ipAdresse,
7 | Wartungsvertrag wartungsvertrag,
8 | boolean hatGrafikkarte) {
9 |
10 | super(leistung, ipAdresse, wartungsvertrag);
11 | this.hatGrafikkarte = hatGrafikkarte;
12 | }
13 |
14 | public boolean isHatGrafikkarte() {
15 | return hatGrafikkarte;
16 | }
17 |
18 | @Override
19 | public double berechneLeistungsaufnahme(int betriebsstundenProTag) {
20 | if (betriebsstundenProTag < 0 || betriebsstundenProTag > 24) {
21 | throw new IllegalArgumentException(
22 | "betriebsstundenProTag muss zwischen 0 und 24 liegen");
23 | }
24 | // Workstation mit starker Grafikkarte verbraucht mehr
25 | double faktor = hatGrafikkarte ? 0.8 : 0.5;
26 | return leistung * faktor * betriebsstundenProTag;
27 | }
28 | }

```

### 4.4 VirtuellerDesktop

Ein virtueller Desktop laeuft z.B. in einer Cloud-Umgebung. Er ist logisch ein Client, aber physisch kein eigenes Geraet im Buero.

```

1 | public class VirtuellerDesktop extends Client {
2 |
3 |     private String providerName;
4 |
5 |     public VirtuellerDesktop(double leistung,

```

```

6  String ipAdresse,
7  Wartungsvertrag wartungsvertrag,
8  String providerName) {
9
10 super(leistung, ipAdresse, wartungsvertrag);
11
12 if (providerName == null || providerName.isBlank()) {
13     throw new IllegalArgumentException(
14         "providerName darf nicht leer sein");
15 }
16 this.providerName = providerName;
17 }
18
19 public String getProviderName() {
20     return providerName;
21 }
22
23 @Override
24 public double berechneLeistungsaufnahme(int betriebsstundenProTag) {
25     if (betriebsstundenProTag < 0 || betriebsstundenProTag > 24) {
26         throw new IllegalArgumentException(
27             "betriebsstundenProTag muss zwischen 0 und 24 liegen");
28     }
29     // Leistungsaufnahme hier eher symbolisch: durchschnittliche
30     // Ressourcennutzung in der Cloud
31     double faktor = 0.7;
32     return leistung * faktor * betriebsstundenProTag;
33 }
34 }

```

## 5 Nutzung in main: Array von Client und teuerster Wartungsvertrag

In der main-Methode erstellen wir ein Array vom Typ `Client[]` und füllen es mit unterschiedlichen konkreten Objekten:

- `clients[0]`: Server
- `clients[1]`: Router
- `clients[2]`: Workstation (ohne Wartungsvertrag, also `null`)
- `clients[3]`: VirtuellerDesktop

Zusätzlich bestimmen wir den **teuersten aktiven Wartungsvertrag** (in Euro pro Monat).

```

1  import java.time.LocalDate;
2
3  public class ClientDemo {
4
5      public static void main(String[] args) {
6
7          Wartungsvertrag vertragServer = new Wartungsvertrag(
8              "WV-1001",
9              LocalDate.of(2024, 1, 1),
10             LocalDate.of(2026, 12, 31),
11             250.0);

```

```

12
13 Wartungsvertrag vertragRouter = new Wartungsvertrag(
14     "WV-2001",
15     LocalDate.of(2023, 6, 1),
16     LocalDate.of(2025, 5, 31),
17     80.0);
18
19 // Workstation hat absichtlich keinen Wartungsvertrag (null)
20 Wartungsvertrag vertragWorkstation = null;
21
22 Wartungsvertrag vertragVDI = new Wartungsvertrag(
23     "WV-3001",
24     LocalDate.of(2024, 3, 1),
25     LocalDate.of(2025, 3, 1),
26     120.0);
27
28 Client[] clients = new Client[4];
29
30 clients[0] = new Server(500.0,
31     "192.168.0.10",
32     vertragServer,
33     8);
34
35 clients[1] = new Router(50.0,
36     "192.168.0.1",
37     vertragRouter,
38     24);
39
40 clients[2] = new Workstation(300.0,
41     "192.168.0.20",
42     vertragWorkstation,
43     true);
44
45 clients[3] = new VirtuellerDesktop(200.0,
46     "10.0.0.5",
47     vertragVDI,
48     "CloudProviderX");
49
50 int betriebsstundenProTag = 8;
51
52 // Polymorphie-Demo und gleichzeitiges Ermitteln des teuersten
53 // aktiven Wartungsvertrags
54 Wartungsvertrag teuersterAktiverVertrag = null;
55
56 for (Client c : clients) {
57     c.einschalten();
58     c.pruefeWartungsvertrag();
59
60     double tagesverbrauch =
61     c.berechneLeistungsaufnahme(betriebsstundenProTag);
62
63     System.out.println(
64         "Tages-Leistungsaufnahme fuer Client "
65         + c.getIpAdresse() + " = " + tagesverbrauch + " Wh");
66
67     System.out.println("-----");
68
69     Wartungsvertrag wv = c.getWartungsvertrag();

```



```

70  if (wv != null && wv.istAktiv()) {
71  if (teuersterAktiverVertrag == null
72  || wv.getMonatlicheKosten()
73  > teuersterAktiverVertrag.getMonatlicheKosten()) {
74  teuersterAktiverVertrag = wv;
75  }
76  }
77  }
78
79  if (teuersterAktiverVertrag != null) {
80  System.out.println(
81  "Teuerster aktiver Wartungsvertrag: "
82  + teuersterAktiverVertrag.getVertragsNummer()
83  + " mit "
84  + teuersterAktiverVertrag.getMonatlicheKosten()
85  + " EUR pro Monat.");
86  } else {
87  System.out.println("Es gibt keinen aktiven Wartungsvertrag.");
88  }
89  }
90  }

```

## 6 Erweiterungsaufgabe: ThinClient

Zur Vertiefung koennen Sie einen weiteren konkreten Typ implementieren:

### Aufgabenstellung

Implementieren Sie eine neue Klasse `ThinClient` als Unterklasse von `Client`.

- Der `ThinClient` soll zusaetzlich ein Attribut `bandbreite` (in Mbit/s, Datentyp `double`) besitzen.
- Im Konstruktor sind die Argumente sinnvoll zu validieren (`bandbreite > 0`).
- Die Methode `berechneLeistungsaufnahme(int betriebsstundenProTag)` soll davon ausgehen, dass `ThinClients` wenig Leistung brauchen, z.B. nur 30% der Nennleistung.
- Erweitern Sie `ClientDemo`, so dass ein `ThinClient` ebenfalls im `Client[]` vorkommt.

## Musterloesung (optional)

```
1 public class ThinClient extends Client {
2
3     private double bandbreite; // in Mbit/s
4
5     public ThinClient(double leistung,
6         String ipAdresse,
7         Wartungsvertrag wartungsvertrag,
8         double bandbreite) {
9
10        super(leistung, ipAdresse, wartungsvertrag);
11
12        if (bandbreite <= 0) {
13            throw new IllegalArgumentException(
14                "bandbreite muss groesser als 0 sein");
15        }
16        this.bandbreite = bandbreite;
17    }
18
19    public double getBandbreite() {
20        return bandbreite;
21    }
22
23    @Override
24    public double berechneLeistungsaufnahme(int betriebsstundenProTag) {
25        if (betriebsstundenProTag < 0 || betriebsstundenProTag > 24) {
26            throw new IllegalArgumentException(
27                "betriebsstundenProTag muss zwischen 0 und 24 liegen");
28        }
29        // ThinClients sind sehr sparsam, z.B. 30% der Nennleistung
30        double faktor = 0.3;
31        return leistung * faktor * betriebsstundenProTag;
32    }
33 }
```

Beispielhafte Erweiterung von ClientDemo (Ausschnitt):

```
1 //... innerhalb von main()
2
3 Wartungsvertrag vertragThinClient = new Wartungsvertrag(
4     "WV-4001",
5     LocalDate.of(2024, 1, 1),
6     LocalDate.of(2025, 1, 1),
7     60.0);
8
9 Client[] clients = new Client[5];
10
11 clients[0] = new Server(500.0,
12     "192.168.0.10",
13     vertragServer,
14     8);
15 //... wie zuvor
16 clients[3] = new VirtuellerDesktop(200.0,
17     "10.0.0.5",
18     vertragVDI,
19     "CloudProviderX");
20 clients[4] = new ThinClient(80.0,
21     "192.168.0.30",
```

```
22 | vertragThinClient ,
23 | 100.0);
```

## 7 Zusammenfassung der Konzepte (Profi-Version)

- **Abstrakte Klasse Client** (mit `protected` Attributen) bildet die gemeinsame Basis fuer alle Client-Typen.
- **Abstrakte Methode** `berechneLeistungsaufnahme`: wird in jeder Unterklasse spezifisch implementiert.
- **Vererbung** („ist eine“): `Server`, `Router`, `Workstation`, `VirtuellerDesktop` und `ThinClient` sind spezielle Arten von `Client`.
- **Komposition** („hat eine“): Ein `Client` hat einen optionalen Wartungsvertrag mit `monatlichenKosten`.
- **Polymorphie ueber Client[]**: Unterschiedliche Client-Typen koennen als `Client` behandelt werden, das konkrete Verhalten ergibt sich zur Laufzeit.
- **Algorithmus**: In `ClientDemo` wird der teuerste aktive Wartungsvertrag ermittelt.
- **Validierung und Exceptions**: Konstruktoren pruefen Eingaben und nutzen `IllegalArgumentException` bei ungueltigen Werten.

## 8 Prüfungsfragen

### Frage 1 – Abstrakte Klasse und Vererbung

Erläutern Sie am Beispiel der Klassen `Client`, `Server` und `Workstation`:

- 1) Warum ist `Client` als **abstrakte Klasse** sinnvoll gewählt?
- 2) Was bedeutet in diesem Zusammenhang die Aussage: „Ein `Server` *ist ein* `Client`“?
- 3) Nennen Sie zwei Vorteile, die sich aus dieser Vererbungsstruktur für die Wartbarkeit oder Erweiterbarkeit des Codes ergeben.

### Frage 2 – Abstrakte Methode und Polymorphie

Betrachten Sie die abstrakte Methode

```
public abstract double berechneLeistungsaufnahme(int betriebsstundenProTag);
```

- 1) Erklären Sie, was eine **abstrakte Methode** ist und warum sie nicht in der Klasse `Client` implementiert wird.
- 2) Beschreiben Sie, wie das Prinzip der **Polymorphie** im Beispiel mit dem Array `Client[] clients` genutzt wird.
- 3) Was passiert zur Laufzeit konkret, wenn in der Schleife über `clients` die Methode `berechneLeistungsaufnahme` aufgerufen wird?

### Frage 3 – „ist eine“- vs. „hat eine“-Beziehung

Im Beispiel gibt es sowohl Vererbung als auch Komposition.

- 1) Erklären Sie den Unterschied zwischen einer „**ist eine**“-Beziehung und einer „**hat eine**“-Beziehung.
- 2) Nennen Sie je ein konkretes Klassenpaar aus dem Beispiel für jede dieser beiden Beziehungsarten und begründen Sie kurz Ihre Wahl.
- 3) Warum ist es fachlich **nicht sinnvoll**, `Wartungsvertrag` von `Client` erben zu lassen?

### Frage 4 – Wartungsvertrag und teuerster Vertrag

Im Beispiel besitzt `Wartungsvertrag` u.a. die Attribute `startDatum`, `endDatum` und `monatlicheKosten`. In `ClientDemo` wird der teuerste aktive Wartungsvertrag ermittelt.

- 1) Beschreiben Sie in eigenen Worten den Algorithmus, mit dem in `ClientDemo` der teuerste aktive Wartungsvertrag gefunden wird.
- 2) Welche Bedingungen müssen erfüllt sein, damit ein Vertrag in die Auswahl des „teuersten aktiven Vertrags“ einbezogen wird?
- 3) Nennen Sie eine mögliche fachliche Erweiterung der Klasse `Wartungsvertrag`, die in einem realen Projekt sinnvoll sein könnte, und begründen Sie kurz.

## 9 Musterlösungen

### Musterlösung zu Frage 1 – Abstrakte Klasse und Vererbung

#### 1. Warum `Client` abstrakt?

- `Client` beschreibt nur einen allgemeinen Netzwerkteilnehmer mit Leistung, IP-Adresse und Wartungsvertrag, ist aber kein konkreter Gerätetyp (in der Praxis spricht man eher von Server, Router, Workstation, `ThinClient` usw.).
- Von `Client` sollen daher **keine direkten Objekte** erzeugt werden, sondern nur von den spezialisierten Unterklassen.
- Die abstrakte Klasse bündelt gemeinsame Attribute und Methoden (`leistung`, `ipAdresse`, `wartungsvertrag`, `einschalten()`, `pruefeWartungsvertrag()`) ohne eine konkrete Ausprägung darzustellen.

#### 2. „Server ist ein Client“

- `Server extends Client` bedeutet: `Server` erbt alle Eigenschaften und Methoden von `Client`.
- Fachlich: Ein `Server` ist ein spezieller Typ von `Client` (Netzwerkteilnehmer) mit zusätzlichen Merkmalen (z.B. `anzahlCpus`) und eigener Implementierung von `berechneLeistungsaufnahme(...)`.
- `Server`-Objekte können überall dort eingesetzt werden, wo ein `Client` erwartet wird (z.B. im `Client[]` Array).

#### 3. Zwei Vorteile der Vererbungsstruktur

- **Code-Wiederverwendung:** Gemeinsame Logik (Validierung im Konstruktor, `einschalten()`, `pruefeWartungsvertrag()`) steht zentral in `Client` und muss nicht in jeder Unterklasse doppelt implementiert werden.
- **Erweiterbarkeit/Polymorphie:** Neue Typen wie `ThinClient` oder `VirtuellerDesktop` können leicht hinzugefügt werden, indem sie `Client` erweitern und nur die spezifischen Teile implementieren. Bestehender Code, der mit Typ `Client` arbeitet (z.B. Schleife über `Client[]`), muss nicht angepasst werden.

### Musterlösung zu Frage 2 – Abstrakte Methode und Polymorphie

#### 1. Abstrakte Methode, warum nicht in `Client` implementiert?

- Eine **abstrakte Methode** hat nur eine Signatur, aber keine Implementierung. Sie zwingt Unterklassen, eine konkrete Implementierung bereitzustellen.
- In `Client` ist nicht klar, wie die Leistungsaufnahme berechnet werden soll, weil sich das Verhalten bei `Server`, `Router`, `Workstation`, `VirtuellerDesktop`, `ThinClient` fachlich unterscheidet.
- Daher wird in `Client` nur festgelegt, dass jede Unterklasse eine Methode `berechneLeistungsaufnahme()` besitzen muss; die konkrete Berechnung erfolgt in den Unterklassen.

#### 2. Polymorphie im `Client[]` Array

- Im Array `Client[]` werden unterschiedliche konkrete Objekte gespeichert (z.B. `Server`, `Router`, `Workstation`, `VirtuellerDesktop`, `ThinClient`).

- Der Code in der Schleife behandelt alle Elemente einheitlich als `Client` und ruft z.B. `c.berechneLeistungsaufnahme(...)` auf.
- Welche Implementierung tatsächlich ausgeführt wird, hängt vom konkreten Objekttyp ab. Das ist Polymorphie: gleicher Methodenname, unterschiedliche Implementierungen.

### 3. Verhalten zur Laufzeit beim Aufruf in der Schleife

- Zur Laufzeit ermittelt die JVM den dynamischen Typ des Objekts, auf das `c` zeigt (z.B. `Server` oder `Router`).
- Dann wird die jeweils passende Überschreibung von `berechneLeistungsaufnahme(...)` in dieser Unterklasse aufgerufen.
- Dadurch ergibt sich je nach konkreter Klasse ein anderes Verhalten, obwohl im Code nur mit dem Typ `Client` gearbeitet wird.

## Musterlösung zu Frage 3 – „ist eine“ vs. „hat eine“Beziehung

### 1. Unterschied „ist eine“ vs. „hat eine“

- **„ist eine“Beziehung:** beschreibt eine Vererbungsbeziehung (Subtyp). Eine spezialisierte Klasse ist eine Ausprägung einer allgemeineren Klasse, z.B. „Server ist ein Client“.
- **„hat eine“Beziehung:** beschreibt eine Kompositions- oder Aggregationsbeziehung. Ein Objekt enthält oder nutzt ein anderes Objekt als Bestandteil, z.B. „Client hat einen Wartungsvertrag“.

### 2. Je ein Klassenpaar aus dem Beispiel

- „ist eine“: `Server` und `Client` → `Server extends Client`. Ein `Server` ist ein spezieller `Client` mit zusätzlichen Eigenschaften und Verhalten.
- „hat eine“: `Client` und `Wartungsvertrag` → Attribut `wartungsvertrag` in `Client`. Ein `Client` hat (optional) einen `Wartungsvertrag`, der Laufzeit und Kosten beschreibt.

### 3. Warum Wartungsvertrag nicht von Client erben?

- Fachlich wäre die Aussage „Ein Wartungsvertrag ist ein Client“ unsinnig, da ein Vertrag kein Netzwerkteilnehmer ist.
- `Wartungsvertrag` besitzt keine Merkmale eines Clients (keine IP-Adresse, keine Leistungsaufnahme), sondern beschreibt nur eine Dienstleistungsvereinbarung.
- Richtig ist daher eine Komposition: Ein `Client` hat einen `Wartungsvertrag`; eine Vererbung würde das Modell verfälschen.

## Musterlösung zu Frage 4 – Wartungsvertrag und teuerster Vertrag

### 1. Algorithmus zum Finden des teuersten aktiven Wartungsvertrags

- Es wird über alle Elemente im Array `Client[]` iteriert.
- Für jeden `Client` wird mit `getWartungsvertrag()` der zugehörige Vertrag geholt.
- Ist der Vertrag `null` oder nicht aktiv (`!istAktiv()`), wird er übersprungen.

- Andernfalls wird geprüft, ob die monatlichen Kosten dieses Vertrags höher sind als die des bisher als teuersten gemerkten Vertrags.
- Ist dies der Fall, wird die Referenz auf diesen Vertrag als `teuersterAktiverVertrag` gespeichert.
- Nach der Schleife wird der teuerste gefundene aktive Vertrag (falls vorhanden) ausgegeben.

## 2. Bedingungen für die Teilnahme an der Auswahl

- Der Vertrag darf nicht `null` sein (es muss ein `Wartungsvertrag` vorhanden sein).
- `istAktiv()` muss `true` liefern, d.h. das aktuelle Datum liegt zwischen `startDatum` und `endDatum` (einschließlich).
- Nur Verträge, die beide Bedingungen erfüllen, können als teuerster aktiver Vertrag in Frage kommen.

## 3. Mögliche fachliche Erweiterung von `Wartungsvertrag`

Beispiele (eines reicht in der Prüfung, jeweils mit Begründung):

- Attribut `serviceLevel` (z.B. „Gold“, „Silber“, „Bronze“) und/oder Reaktionszeiten im Fehlerfall.  
Begründung: Service-Level-Agreements (SLA) sind in echten Wartungsverträgen üblich und definieren Qualitätszusagen.
- Attribut `kuendigungsfristInTagen` und Methode `berechneKuendigungsdatum()`.  
Begründung: Kunden müssen wissen, bis wann ein Vertrag ordentlich gekündigt werden kann.
- Methode `berechneGesamtkosten()`, die auf Basis von Laufzeit und monatlichen Kosten die Gesamtkosten ermittelt.  
Begründung: Für kaufmännische Auswertungen sind die Gesamtkosten relevant.

## 10 UML-Klassendiagramm (ASCII)

Das folgende UML-Diagramm im ASCII-Format beschreibt die Struktur der Klassenhierarchie rund um die abstrakte Klasse `Client`, die konkreten Unterklassen sowie die Komposition mit `Wartungsvertrag`.

### UML-ASCII-Diagramm

```
+-----+
| <<abstract>> |
| Client |
+-----+
| - leistung: double |
| - ipAdresse: String |
| - wartungsvertrag: |
| Wartungsvertrag |
+-----+
| + Client(leistung, |
| ipAdresse, |
| wartungsvertrag) |
| + getLeistung(): double |
| + getIpAdresse(): String |
| + getWartungsvertrag(): |
| Wartungsvertrag |
| + einschalten(): void |
| + pruefeWartungsvertrag(): |
| void |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int)|
| : double {abstract} |
+-----+

+-----+
| Server |
+-----+
| - anzahlCpus: int |
+-----+
| + Server(leistung, |
| ipAdresse, |
| wartungsvertrag, |
| anzahlCpus) |
| + getAnzahlCpus(): int |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int)|
| : double |
+-----+
```



```

+-----+
| Router |
+-----+
| - anzahlPorts: int |
+-----+
| + Router(leistung, |
| ipAdresse, |
| wartungsvertrag, |
| anzahlPorts) |
| + getAnzahlPorts(): int |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int)|
| : double |
+-----+

+-----+
| Workstation |
+-----+
| - hatGrafikkarte: boolean |
+-----+
| + Workstation(leistung, |
| ipAdresse, |
| wartungsver- |
| trag, |
| hatGrafik- |
| karte) |
| + isHatGrafikkarte(): |
| boolean |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int)|
| : double |
+-----+

+-----+
| VirtuellerDesktop |
+-----+
| - providerName: String |
+-----+
| + VirtuellerDesktop( |
| leistung, |
| ipAdresse, |
| wartungsvertrag, |
| providerName) |
| + getProviderName(): |
| String |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int)|
| : double |
+-----+

```

```

+-----+
| ThinClient |
+-----+
| - bandbreite: double |
+-----+
| + ThinClient(leistung, |
| ipAdresse, |
| wartungsver- |
| trag, |
| bandbreite) |
| + getBandbreite(): double |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int)|
| : double |
+-----+

+-----+
| Wartungsvertrag |
+-----+
| - vertragsNummer: String |
| - startDatum: LocalDate |
| - endDatum: LocalDate |
| - monatlicheKosten: double |
+-----+
| + Wartungsvertrag(vertrags- |
| Nummer, startDatum, |
| endDatum, monatliche- |
| Kosten) |
| + getVertragsNummer(): String |
| + getMonatlicheKosten(): double |
| + istAktiv(): boolean |
+-----+

```